



7

8

1200 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

التوابع 2

د. مدحت الصوص

د. فراس ضعيف



برمجة ا

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 4/1/2026

التصريح عن التوابع

هل يمكن استدعاء تابع قبل تعريفه؟

لدينا الكود الآتي الذي يسبب خطأ عند محاولة تنفيذه

```
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    cout << sum(a, b) << endl;
    return 0;
}
int sum(int a, int b) {
    return a + b;
}
```



لماذا يسبب الكود السابق خطأ عند تنفيذه وما الحل؟

في لغة C++ يقرأ الـ compiler الكود بشكل تسلسلي (من الأعلى إلى الأسفل)، لذلك عند استدعاء تابع يجب أن يكون المترجم قد تعرّف عليه مسبقاً، وإلا سيظهر خطأ لأنه لم يتعرّف عليه بعد، وهذا ما حصل في الكود السابق. يمكن تبسيط الفكرة بالقول إن الـ compiler في C++ يتعامل مع الكود وكأنه يقرأه مرة واحدة، لذلك يجب التصريح عن التابع قبل استخدامه.

أما في بعض اللغات الأخرى مثل java و C# فيتم التعرف على التوابع ضمن الملف بشكل كامل قبل التنفيذ، لذلك يمكن استدعاء التابع حتى لو كُتب تعريفه لاحقاً.

للسهولة غالباً نكتب تابع main في نهاية البرنامج، ولكن في بعض الحالات نحتاج إلى وجود تابعين يستدعي كل منهما الآخر. عندها لا يكفي كتابة تابع قبل الآخر، لأن كل تابع يحتاج أن يكون معرفاً قبل استدعائه.

الحل الأمثل هنا هو التصريح عن التابع (Function Prototype) قبل كتابة تعريفه الكامل، وذلك لإخبار الـ compiler بوجود هذا التابع قبل استخدامه.

```
#include <iostream>
using namespace std;
int sum(int a, int b);
int main() {
    int a, b;
    cin >> a >> b;
    cout << sum(a, b) << endl;
    return 0;
}
int sum(int a, int b) {
    return a + b;
}
```

- عند التصريح عن التابع نحن نخبر الـ compiler بوجود تابع يعيد قيمة من النوع int ويستقبل عددين من النوع int، وذلك قبل كتابة مجموعة تعليماته.
- يمكن أثناء التصريح كتابة نوع الوسطاء وعددهم فقط دون كتابة أسمائهم، كما نرى:
- وجود الفاصلة المنقوطة بعد قوسي التابع (); يعني أننا نصريح عن التابع فقط دون كتابة تعليماته، وسيتم تعريفه لاحقاً في البرنامج.

تمرير المصفوفات للتابع

1. تمرير مصفوفة أحادية لتابع:

```
#include <iostream>
using namespace std;
void print(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << endl;
}
int main() {
    int array[3] = { 0, 1, 5 };
    print(array, 3);
    return 0;
}
```

تترك فارغة لأن التابع
يستقبل المصفوفة
نفسها وليس نسخة منها.

التابع print يملك وسيطين:

1. مصفوفة نوعها int

2. n طول المصفوفة (عدد عناصرها)

عند استدعاء print داخل main نضع اسم المصفوفة فقط دون وضع [] أو وضع أي قيمة بداخلها

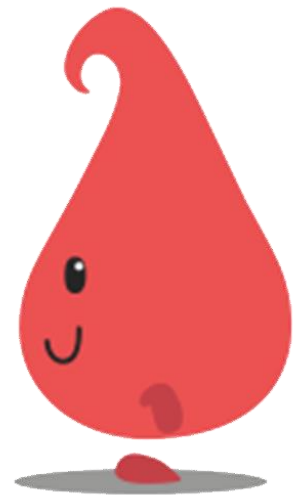
عند إرسال مصفوفة إلى تابع، لا تُنسخ المصفوفة كاملة، بل يُمرّر عنوان أول عنصر منها، لذلك التابع يتعامل مع نفس المصفوفة الأصلية.

2. تمرير مصفوفة ثنائية لتابع:

تذكير: لا تُخزن المصفوفات الثنائية في الذاكرة RAM على شكل أسطر وأعمدة كما نراها نحن، بل تُخزن على شكل خانات متتالية في الذاكرة، وذلك لأنها تعد سطر واحد طويل، لذلك يتم تخزين عناصر المصفوفة الثنائية بشكل متتال حسب الأسطر.

مما سبق نستنتج أنه عند إرسال مصفوفة ثنائية إلى تابع يجب تحديد البعد الثاني (عدد الأعمدة)، وذلك لتمكين الـ compiler من معرفة عدد العناصر التي يجب تجاوزها للانتقال إلى السطر التالي. فلو كان لدينا مصفوفة تحوي 4 أسطر و 3 أعمدة فإنها تُخزن في الذاكرة RAM على شكل 12 خانة متتالية. وعندما يعرف الـ compiler عدد الأعمدة (3)، يمكنه تحديد مكان بداية كل سطر وطريقة الوصول الصحيحة إلى عناصر المصفوفة. مثال:

```
#include <iostream>
using namespace std;
void print(int arr[][2], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < 2; j++)
            cout << arr[i][j];
        cout << endl;
    }
}
void getarray(int arr[][2], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < 2; j++)
            cin >> arr[i][j];
    }
}
int main() {
    int array[3][2];
    getarray(array, 3);
    print(array, 3);
    return 0;
}
```



إذاً: لا نحدد عدد الصفوف في تعريف وسيط التابع لأن المصفوفة تمرر كعنوان، لكن يجب تحديد عدد الأعمدة حتى يستطيع المترجم حساب مكان العنصر التالي.

- عند تمرير مصفوفة 3D إلى تابع ما نحدد البعدين الثاني والثالث في وسيط التابع
- عند تمرير مصفوفة 4D إلى تابع ما نحدد البعدين الثاني والثالث والرابع في وسيط التابع

المتحولات العامة والمتحولات المحلية:

ال scope هو النطاق الذي يحدد البداية والنهاية لكل تابع ويتحدد بالقوسين {} ولا يقتصر على التوابع فقط بل كل زوج من الأقواس {} ينشئ نطاقاً جديداً مثل if, for, while..

يمكن تبسيط الذاكرة في البرنامج إلى قسمين رئيسيين:
Stack: يمتلك هذا الجزء حجماً صغيراً يتعلق بلغة البرمجة ويقدر عادة بعدة ميغابايتات
Heap: يضم هذا الجزء غالبية الذاكرة RAM ويزداد عند تركيب ذاكرة إضافية للحاسب

1. المتحولات المحلية local variables

تدعى المتحولات التي يتم تعريفها داخل التوابع بالمتحولات المحلية ويتم حجز المساحة المخصصة لها في الذاكرة RAM ضمن الجزء المسمى stack وتعتبر وسطاء أي تابع متحولات محلية لهذا التابع. تبدأ حياة المتحولات المحلية مع بداية التابع وتنتهي عند نهايته

2. المتحولات العامة global variables

هي متحولات تُعرّف خارج جميع التوابع، ويمكن استخدامها داخل أي تابع في البرنامج. كما يمكن استخدامها عندما يكون حجم المتحول كبيراً ولا يتسع في الـ stack يمكن الوصول لهذه المتحولات من أي تعليمة ضمن أي تابع من البرنامج، ويتم حجز مساحة خاصة بهذه المتحولات، ضمن الجزء المسمى heap. وتسمح هذه المتحولات العامة بالتواصل بين التوابع بطريقة مختلفة عن المتحولات المستخدمة كوسطاء للتابع

تبدأ حياة المتحولات العامة مع بداية البرنامج وتنتهي مع نهايته لأنها معرفة خارج التوابع أي خارج أي scope

```
#include <iostream>
using namespace std;
int a = 2, b = 1;
int main() {
    int s = 5;
    cout << s << endl;
    cout << a + b << endl;
    return 0;
}
```

OUTPUT

```
5
3
```

هنا لدينا a, b هي متحولات عامة، بينما s هو متحول محلي ضمن التابع main()

المتحولات العامة	المتحولات المحلية	
في كل البرنامج	ضمن النطاق المعرفة بداخله	مجال الاستخدام
heap	stack	مكان التخزين

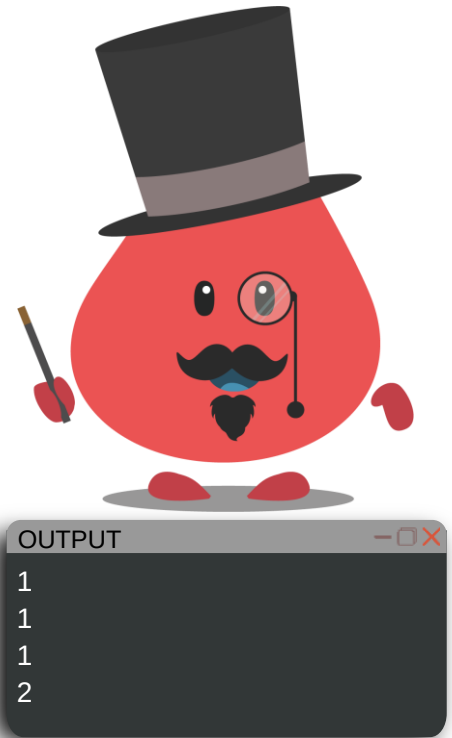
يمكن أن يوجد متحول عام ومتحول محلي يحملان الاسم نفسه، وعند استخدام الاسم داخل التابع يتم التعامل مع المتحول المحلي تلقائياً لأنه ضمن النطاق الحالي. توجد طرق للوصول إلى المتحول العام لن نتطرق إليها الآن

المتغيرات من نوع static

يتم إنشاء هذا النوع من المتحولات مرة واحدة فقط، ويمكن أن تتغير قيمته عدة مرات، حيث توجد منه نسخة واحدة فقط في الذاكرة.

يشبه المتحول المحلي من حيث تعريفه داخل scope معين وعدم إمكانية استخدامه خارج هذا النطاق، إلا أنه لا يُحذف عند انتهاء الـ scope الخاص به، مما يسمح بالاحتفاظ بقيمته وإعادة استخدامها عند تنفيذ هذا الـ scope مرة أخرى مثال:

```
#include <iostream>
using namespace std;
int fun() {
    int n = 0;
    n = n + 1;
    return n;
}
int funstatic() {
    static int n = 0;
    n = n + 1;
    return n;
}
int main() {
    cout << fun() << endl;
    cout << fun() << endl;
    cout << funstatic() << endl;
    cout << funstatic() << endl;
    return 0;
}
```



عند استدعاء التابع fun يتم في كل مرة تعريف متحول جديد باسم n وإسناد القيمة 0 له، ثم زيادته بمقدار 1، لذلك يعيد التابع دائماً القيمة 1 عند كل استدعاء.

أي أن استدعاء التابع fun للمرة الأولى يطبع 1، وعند استدعائه مرة أخرى يتم إنشاء متحول جديد من جديد بقيمة 0 ثم زيادته، فيطبع أيضاً 1.

أما في التابع funstatic فيتم تعريف المتحول n مرة واحدة فقط عند أول استدعاء للتابع، ويبقى محتفظاً بقيمته بعد انتهاء التنفيذ.

لذلك عند الاستدعاء الأول تصبح قيمة n تساوي 1 ويتم طباعة 1، وعند الاستدعاء مرة أخرى لا يُعاد تعريف المتحول بل تستخدم قيمته السابقة، فتزداد إلى 2 ويتم طباعة 2

التحميل الزائد function overloading

هل يمكننا تعريف تابعين يمتلكان الاسم نفسه؟

نعم ولكن بشروط منها:

1- اختلاف بنوع الوسطاء أو بعددها بين التوابع التي تحمل نفس الاسم

2- ألا يكون الاختلاف في نوع القيمة المعادة فقط

لا يمكن التمييز بين التوابع اعتماداً على نوع القيمة المعادة فقط، بل يجب وجود اختلاف في عدد الوسطاء أو نوعهم. مثال: يمكننا تعريف تابعين كلاهما يحملان اسم sum لكن تابع يقوم بجمع متحولين وتابع آخر يقوم بجمع ثلاث متحولات

```
#include <iostream>
using namespace std;
int sum(int x, int y) {
    cout << "This is fun 1: ";
    return x + y;
}
int sum(int x, int y, int z) {
    cout << "This is fun 2: ";
    return x + y + z;
}
int main() {
    cout << sum(1, 2) << endl;
    cout << sum(1, 2, 3) << endl;
}
```

OUTPUT

```
This is fun 1: 3
This is fun 2: 6
```

مثال: تعريف تابعين كلاهما يحملان اسم sum ولهما نفس عدد الوسطاء لكن باختلاف نوع كل وسيط

```
#include <iostream>
using namespace std;
int sum(float x, int y) {
    cout << "This is fun 1 ";
    return x + y;
}
int sum(int x, float y) {
    cout << "This is fun 2 ";
    return x + y;
}
int main() {
    cout << sum(1.5, 2) << endl;
    cout << sum(1, 2.0) << endl;
}
```

OUTPUT

```
This is fun 1: 3.5
This is fun 2: 3
```

عندما يكون الاستدعاء على الشكل sum(2, 3) هنا يظهر error لأن المترجم لا يعرف أي التابعين يختار

في حال كان الاستدعاء sum(1.0, 3) يستدعي التابع الأول

في حال كان الاستدعاء sum(2, float(4)) يستدعي التابع الثاني

Casting

الـ Casting هو تحويل مؤقت لقيمة من نوع بيانات إلى نوع آخر.

- نستخدمه عند الحاجة للتعامل مع القيمة بنوع مختلف عن نوعها الأصلي.
 - يُستخدم أيضاً للحصول على نتائج صحيحة في العمليات الحسابية، خاصة عند القسمة بين أعداد صحيحة.
- عند استخدام casting على متحول ما فإن نوعه الأصلي لا يتغير، وإنما يتم التعامل معه مؤقتاً بالنوع الجديد فقط أثناء تنفيذ العملية. وكأننا وضعنا المتحول في علبة جديدة بشكل مؤقت لاستخدامه، ثم يعود بعد انتهاء العملية إلى نوعه الأصلي دون أي تغيير.
- مثال:

```
int a = 5, b = 2;
double result = (double)a / b;
```

بدون casting ستكون النتيجة 2

لكن بعد التحويل إلى double تصبح النتيجة 2.5

Suffix & Prefix

ما الفرق بين ++x و x++

(Suffix) ++x ترجع القيمة القديمة لـ x ثم تزيدها بمقدار 1

مثال:

```
int x = 5;
cout << x++ << endl; //prints 5
cout << x << endl;   //prints 6
```

(Prefix) ++x تزيدها قيمة x بمقدار 1 ثم ترجع القيمة الجديدة

مثال:

```
int x = 5;
cout << ++x << endl; //prints 6
cout << x << endl;   //prints 6
```

مثال للمقارنة:

```
int x = 3;
y = x++; //y=3, x=4
y = ++x; //x=5, y=5
```

الفرق بينهما يظهر فقط عند استخدام المتحول داخل تعبير أو إسناد.

الأمر ينطبق على --x و x--



تعيين قيمة ابتدائية للوسطاء:

يمكن إعطاء قيمة ابتدائية لوسيط لتكون قيمة الافتراضية له في حال لم نقوم بوضع أي قيمة مكان هذا الوسيط عند استدعاء التابع، ولكن عند وضع قيمة لهذا الوسيط أثناء استدعاء التابع فإن الوسيط يأخذ هذه القيمة ويتجاهل القيمة الابتدائية

مثال

```
1. #include <iostream>
2. using namespace std;
3. int add(int a, int b = 2) {
4.     return a + b;
5. }
6. int main() {
7.     cout << add(12) << endl;
8.     cout << add(12, 4) << endl;
9.     return 0;
10. }
```

OUTPUT

```
14
16
```

بالنظر للسطر 7 تم تمرير قيمة للوسيط a ولم يتم تمرير قيمة للوسيط b وبالتالي سيتم اعتبار القيمة الافتراضية 2 على أنها قيمة b وسيتم جمعها مع 12 ليعيد التابع 14

أما في السطر 8 تم تمرير القيمة 4 للوسيط b بالتالي سيتجاهل القيمة الافتراضية للوسيط b ومنه فإنه سيعيد 16 الآن أصبح بإمكاننا كتابة تابع يقوم إما بجمع عددين أو ثلاثة أعداد من خلال إعطاء المتحول الثالث قيمة ابتدائية وهي الصفر

مثال آخر:

```
#include <iostream>
using namespace std;
int add(int x, int y, int z = 0) {
    return x + y + z;
}
int main() {
    cout << add(2,3) << endl;
    cout << add(2,3,4) << endl;
    return 0;
}
```

OUTPUT

```
5
9
```

ملاحظة: في حال تم إسناد قيمة افتراضية لوسيط ما فإنه يجب إعطاء قيمة افتراضية لكل الوسطاء التي تليه عند التصريح عن التابع أي

غير مسموح:

```
int sum(int x, int y = 0, int z)
```

مسموح:

```
int sum(int x, int y, int z = 0)
int sum(int x, int y = 0, int z = 0)
```